AIO 2022 — Problem 5: Composing Pyramids — O(N²) Brute Force (No DP)

This version drops the `bestUp[]` / `bestDown[]` lookup tables from the optimized solution and instead recomputes everything **from scratch, independently, for every position**. It's `O(N²)` instead of `O(N)`, which is enough for Subtasks 1–2 (and a good first solution to get working before optimizing), but it's a clean stepping stone toward the DP because the *idea* is identical — only the "reuse work across positions" part is removed.

1. The two quantities we still need

Same as before:

- `up[i]` = length of the longest run of consecutive ascending integers ending **at** position `i`.
- `down[i]` = length of the longest run of consecutive descending integers starting **at** position `i`.
- Best pyramid through peak `i`: `2 * min(up[i]-1, down[i]-1) + 1`.

The only thing changing is *how* we compute `up[]` and `down[]`.

2. The greedy matching lemma (this replaces the DP)

To compute `up[i]`, we need: what's the largest `r` such that values `P[i]-r, ..., P[i]-1` can be found, in that order, at strictly increasing positions, all before position `i`?

Claim: scanning backward from `i-1`, always accepting the *first* (i.e. rightmost) occurrence of whatever value you currently need, is optimal.

Why: suppose you need value `x` somewhere before position `p`. Any occurrence of `x` works for "is `x` present" — but the occurrence you pick becomes the new right-boundary `p'` for finding `x-1`. Picking the rightmost (latest) valid occurrence of `x` makes `p'` as large as possible, which only ever gives *more* room — never less — to find `x-1` before it. So rightmost-first can never do worse than any other choice. (Mirror argument for `down[i]`: scanning forward, always take the *leftmost* occurrence, which leaves maximum room on the right for what follows.)

This means each of `up[i]` and `down[i]` can be computed with a **single, self-contained scan** — no table of "best result so far" needed, and no dependency on any other position's answer.

3. The algorithm

For every position `i` (independently):

- Walk backward from `i-1` to `1`. Keep a `target` counter starting at `P[i]-1`. Every time you pass a cell equal to `target`, that's a match — increment your count and decrement `target`. Stop early if `target` drops below 1.
- Symmetrically, walk forward from `i+1` to `N` for `down[i]`.

Since each scan only ever moves in one direction and never backtracks, each one is `O(N)` in the worst case — giving `O(N²)` total over all `i`. No recursion, no memo table, nothing referencing any other position's stored result.

4. Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N;
    scanf("%d", &N);
    vector<int> P(N + 1);
    for (int i = 1; i <= N; i++) scanf("%d", &P[i]);

    vector<int> up(N + 1), down(N + 1);

    // up[i]: longest ascending consecutive run ending exactly at i.
    // Recomputed independently for every i -- a plain backward scan,
    // no stored results from any other position are used.
    for (int i = 1; i <= N; i++) {
        int target = P[i] - 1; // the value we're currently hunting for
        int r = 0;
        for (int j = i - 1; j >= 1 && target >= 1; j--) {
            if (P[j] == target) {  // first match found scanning backward
                r++;               // = the RIGHTMOST occurrence of target
                target--;          // now hunt for the next value down
            }
        }
        up[i] = r + 1; // +1 to count position i itself
```

```
    }

    // down[i]: longest descending consecutive run starting exactly at i.
    // Mirror image: forward scan, accept the LEFTMOST occurrence each time.
    for (int i = 1; i <= N; i++) {
        int target = P[i] - 1;
        int r = 0;
        for (int j = i + 1; j <= N && target >= 1; j++) {
            if (P[j] == target) {
                r++;
                target--;
            }
        }
        down[i] = r + 1;
    }

    // Combine: try every position as the peak.
    int bestLen = 1; // a lone key is always a valid pyramid
    for (int i = 1; i <= N; i++) {
        int radius = min(up[i] - 1, down[i] - 1);
        bestLen = max(bestLen, 2 * radius + 1);
    }

    printf("%d\n", N - bestLen);
    return 0;
}
```

5. Correctness check (Sample 1)

`P = [2,1,3,4,2,1]`. Running the scans above gives `up = [1,1,2,3,2,1]` and `down = [2,1,3,1,2,1]` — identical to the DP version's tables, confirming the greedy-scan approach computes exactly the same values, just by brute force. Best peak is position 3 (radius 1, length 3) → answer `6 - 3 = 3`. ✅

6. Complexity & where this lands you

- Time: **O(N²)** — comfortably fine for Subtask 1 (N≤100) and Subtask 2 (N≤1000); too slow for Subtask 5 (N up to 100,000, where $O(N^2) \approx 10^{10}$).
- Space: **O(N)**.

7. The natural next question for students

Once this version is working, a good prompt is: *“In the backward scan for `up[i]`, you’re re-deriving the rightmost-occurrence chase from scratch every single time — but didn’t you already do almost this exact walk when computing `up[i-1]`, `up[i-2]`, etc.? What if you kept a running record of the best chain length seen so far for each value, instead of rescanning?” `That question is exactly what motivates the` bestUp[] `/` bestDown[]` arrays in the O(N) DP solution — same greedy correctness argument, just with the repeated work cached instead of redone.